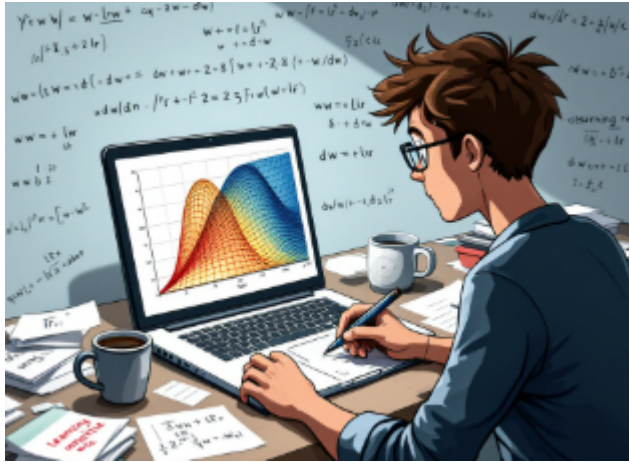


Implementing Linear Regression with Gradient Descent from Scratch



In this notebook, I have implement **Linear Regression using Gradient Descent from scratch** to understand its working and mathematical foundation. I have started with a simple 2D case and then extend it to multi-dimensional data.

To validate the approach, I will compare the results with **Scikit-Learn's** `LinearRegression` and evaluate the model's performance.

What is Gradient Descent?

Gradient Descent is an optimization algorithm used to minimize the cost function in machine learning models by iteratively updating model parameters. It is widely used in linear regression, logistic regression, and neural networks.

Why Do We Need Gradient Descent?

In machine learning, we often have a cost function (or loss function) that tells us how well our model is performing. Our goal is to minimize this cost function by finding the best parameters (weights, biases, or coefficients) that make predictions as accurate as possible.

Notebook Contains:

1. Mathematical formulas for linear regression with gradient descent.
2. Implement linear regression for both 2D and multi-dimensional data.
3. Compared the implementation with `sklearn` `LinearRegression`.

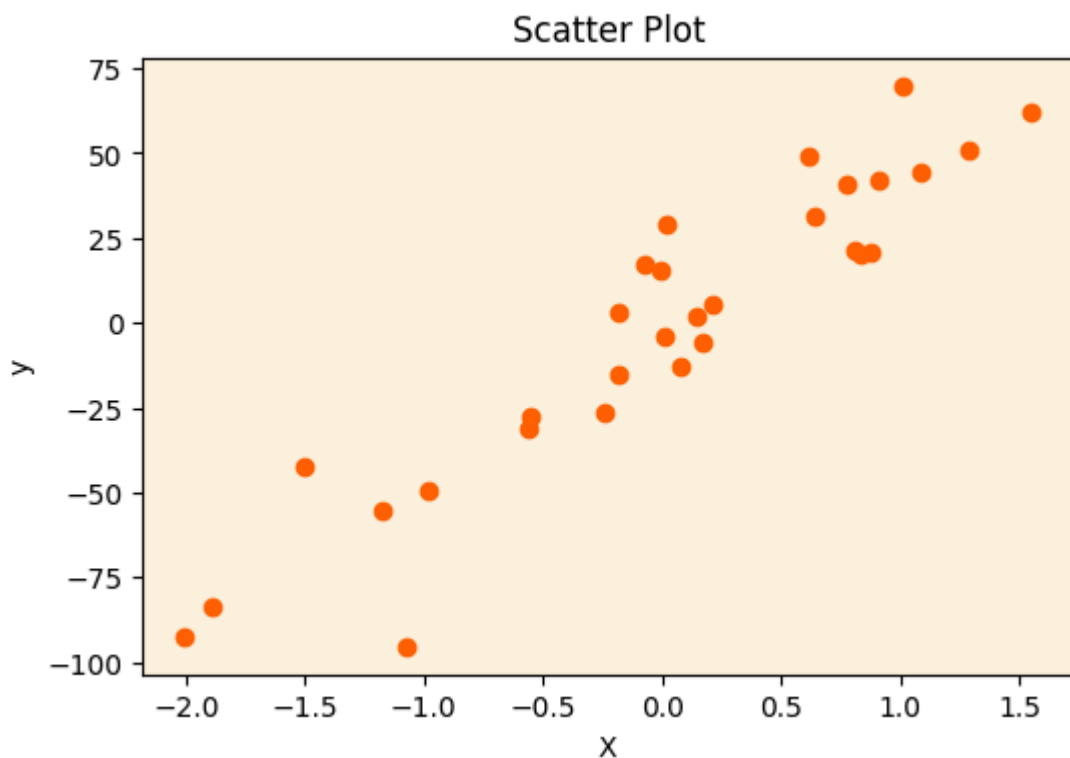
Let's get started!

```
In [1]: #Importing Sklearn Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import make_regression, load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
In [2]: # creating the regression dataset
X, y = make_regression(n_samples=30, n_features=1, noise=15, random_state=14)
```

```
In [3]: # Plotting the scatter plot of the generated regression dataset
fig, ax = plt.subplots(figsize=(6,4))
ax.scatter(X, y, color = '#ff6200')
ax.set_xlabel('X')
ax.set_ylabel('y')
ax.set_title('Scatter Plot')
ax.set_facecolor('#fcf0dc')
plt.show()
```



```
In [4]: # train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=
```

Step-by-Step Derivation of Gradient Descent for $y = mx + b$

where:

- y is the predicted value.
- m is the slope of the line.
- x is the input feature.
- b is the y-intercept.

The cost function (**mean squared error**) for linear regression is given by:

$$J(m, b) = \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + b))^2$$

- The partial derivatives of the cost function with respect to m and b are

$$\frac{\partial J}{\partial m} = -2 \cdot \frac{1}{n} \sum_{i=1}^n x_i (y_i - (mx_i + b))$$

$$\frac{\partial J}{\partial b} = -2 \cdot \frac{1}{n} \sum_{i=1}^n (y_i - (mx_i + b))$$

- These derivatives are used to update the parameters m and b iteratively until the cost function converges to a minimum value.
- updating m and b

$$m := m - \alpha \frac{\partial J}{\partial m}$$

$$b := b - \alpha \frac{\partial J}{\partial b}$$

- where α is the learning rate.

```
In [5]: # Class for Simple Gradient Descent
class SimpleGD:
    def __init__(self, learning_rate=0.001, epochs=100):
        self.m = None
        self.b = None
        self.learning_rate = learning_rate
        self.epochs = epochs

    def fit(self, X_train, y_train):
        # Initialize parameters
        self.m = np.ones(1)
        self.b = np.zeros(1)

        for _ in range(self.epochs):
            y_hat = (self.m * X_train.ravel()) + self.b # y = m*X + b

            d_m = -2 * np.mean(X_train.ravel() * (y_train - y_hat)) # Partial derivative w.r.t m
            d_b = -2 * np.mean(y_train - y_hat) # Partial derivative w.r.t b
```

```

    # updating m and b
    self.m = self.m - (self.learning_rate * d_m)
    self.b = self.b - (self.learning_rate * d_b)

    # Predictions
    def predict(self,X_test):
        return (self.m * X_test) + self.b

```

Model Performance for 20 Epochs with Learning Rate = 0.01

```

In [6]: simple_gd = SimpleGD(learning_rate=0.01,epochs=20)
simple_gd.fit(X_train,y_train)
y_pred = simple_gd.predict(X_test)
print(f"RMSE --> {np.sqrt(mean_squared_error(y_test,y_pred))}")
print(f"R2 Score --> {r2_score(y_test,y_pred)}")

```

RMSE --> 48.49943702444557

R2 Score --> 0.26654096485018797

After training the linear regression model for 20 epochs with a learning rate of 0.01, the evaluation metrics are:

Root Mean Squared Error (RMSE): **48.4994**

R2 Score: **0.2665**

The R2 score is quite low, indicating that the model is not fitting the data well. To improve performance, I will tune the model by increasing the epochs to 100 and setting the learning rate to 0.1 to observe its impact on accuracy.

Tuned Model Performance for 100 Epochs with Learning Rate = 0.1

```

In [7]: simple_gd = SimpleGD(learning_rate=0.1,epochs=100)
simple_gd.fit(X_train,y_train)
y_pred = simple_gd.predict(X_test)
print(f"RMSE --> {np.sqrt(mean_squared_error(y_test,y_pred))}")
print(f"R2 Score --> {r2_score(y_test,y_pred)}")

```

RMSE --> 23.962434739416324

R2 Score --> 0.8209541108171734

After increasing the epochs to 100 and setting the learning rate to 0.1, the evaluation metrics are:

Root Mean Squared Error (RMSE): **23.9624**

R2 Score: **0.8209**

The R2 score has improved significantly from **0.2665 to 0.8209**, indicating that the model is now fitting the data much better. This demonstrates that increasing the number of epochs and adjusting the learning rate can have a substantial impact on performance.

Scikit-Learn's Linear Regression Model

```
In [8]: lr = LinearRegression()
lr.fit(X_train,y_train)
y_pred = lr.predict(X_test)
print(f"RMSE --> {np.sqrt(mean_squared_error(y_test,y_pred))}")
print(f"R2 Score --> {r2_score(y_test,y_pred)}")
```

RMSE --> 23.962431751571376

R2 Score --> 0.8209541554671707

To compare the performance of our custom Gradient Descent-based Linear Regression, I used Scikit-Learn's LinearRegression model. The evaluation results are:

Root Mean Squared Error (RMSE): **23.9624**

R2 Score: **0.8209**

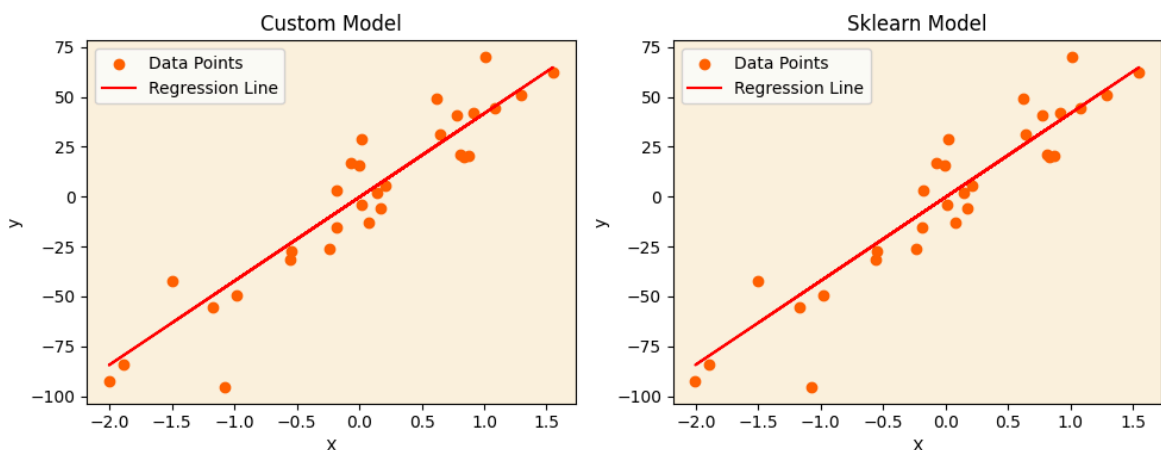
These results are almost identical to our custom implementation, validating that our gradient descent approach is working correctly.

```
In [9]: fig,ax = plt.subplots(nrows=1,ncols=2,figsize=(10,4))

# Custom Model
ax[0].scatter(X,y,color = '#ff6200',label='Data Points')
ax[0].plot(X_train,simple_gd.predict(X_train),color = 'red',label='Regression Li
ax[0].set_xlabel('X')
ax[0].set_ylabel('y')
ax[0].set_title('Custom Model')
ax[0].set_facecolor('#fcf0dc')
ax[0].legend()

# Sklearn model
ax[1].scatter(X,y,color = '#ff6200',label='Data Points')
ax[1].plot(X_train,lr.predict(X_train),color = 'red',label='Regression Line')
ax[1].set_xlabel('X')
ax[1].set_ylabel('y')
ax[1].set_title('Sklearn Model')
ax[1].set_facecolor('#fcf0dc')
ax[1].legend()

plt.tight_layout()
plt.show()
```



Plotted the scatter plot and best-fit line for custom model and Scikit-Learn's LinearRegression model.

The results show that the scatter plot and regression line for both models are identical, confirming that our custom implementation performs just as well as Scikit-Learn's built-in model.

Gradient Descent Derivation for Linear Regression in n Dimensions

For multiple features (n-dimensional input), the prediction is given by:

$$\hat{y}_i = w_1x_{i1} + w_2x_{i2} + \dots + w_nx_{in} + b$$

$$\hat{y}_i = \mathbf{w}^T \mathbf{x}_i + b$$

- in vector form
- The cost function (**mean squared error**) for linear regression is given by:

$$J(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

$$J(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m (y_i - (\mathbf{w}^T \mathbf{x}_i + b))^2$$

- Partial Derivatives w.r.t $\mathbf{w}$

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \mathbf{w}} (y_i - \mathbf{w}^T \mathbf{x}_i - b)^2$$

$$= \frac{1}{m} \sum_{i=1}^m 2(y_i - \mathbf{w}^T \mathbf{x}_i - b) \cdot (-\mathbf{x}_i)$$

$$= -\frac{2}{m} \sum_{i=1}^m \mathbf{x}_i (y_i - \mathbf{w}^T \mathbf{x}_i - b)$$

- Partial Derivatives w.r.t b

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial b} (y_i - \mathbf{w}^T \mathbf{x}_i - b)^2$$

$$= \frac{1}{m} \sum_{i=1}^m 2(y_i - \mathbf{w}^T \mathbf{x}_i - b) \cdot (-1)$$

$$= -\frac{2}{m} \sum_{i=1}^m (y_i - \mathbf{w}^T \mathbf{x}_i - b)$$

- Updating w and b

$$\mathbf{w} = \mathbf{w} - \alpha \nabla_{\mathbf{w}} J$$

$$b = b - \alpha \nabla_b J$$

```
In [10]: # Gradient Descent
class GradientDescent:
    def __init__(self, learning_rate=0.01, epochs = 100):
        self.coef_ = None
        self.intercept_ = None
        self.learning_rate = learning_rate
        self.epochs = epochs

    def fit(self, X_train, y_train):
        self.intercept_ = np.zeros(1) # assigning b to 0
        self.coef_ = np.ones(X_train.shape[1]) # assigning w to 1

        for _ in range(self.epochs):
            y_hat = np.dot(X_train, self.coef_) + self.intercept_ # y predictions

            d_intercept = -2 * np.mean(y_train - y_hat) # Partial Derivatives w.r.t b
            d_coef = -2 * np.dot((y_train - y_hat), X_train) / X_train.shape[0] # Partial

            # updating w and b
            self.intercept_ = self.intercept_ - (self.learning_rate * d_intercept)
            self.coef_ = self.coef_ - (self.learning_rate * d_coef)

    def predict(self, X_test):
        return np.dot(X_test, self.coef_) + self.intercept_
```

I have coded the Gradient Descent Derivation for Linear Regression in n Dimensions.

Let's test our model with the Diabetes dataset from `sklearn.datasets` and compare the results with Scikit-Learn's Linear Regression model.

```
In [11]: data = load_diabetes()
df = pd.DataFrame(data.data, columns=data.feature_names)
```

```
df['target'] = data.target
```

```
In [12]: df.head()
```

```
Out[12]:
```

	age	sex	bmi	bp	s1	s2	s3	s4
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592

```
In [13]: X = df.drop('target',axis=1)
y = df['target']
```

```
In [14]: X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=
```

Model Performance for 100 Epochs with Learning Rate = 0.01

```
In [15]: dg = GradientDescent(learning_rate = 0.01,epochs = 100)
dg.fit(X_train,y_train)
y_pred = dg.predict(X_test)
print(f"RMSE --> {np.sqrt(mean_squared_error(y_test,y_pred))}")
print(f"R2 Score --> {r2_score(y_test,y_pred)}")
```

RMSE --> 81.76137291680337

R2 Score --> -0.10799988491152668

After training the Gradient Descent model with a learning rate of 0.01 for 100 epochs, we obtained:

Root Mean Square Error (RMSE): **81.76**, indicating the average prediction error.

R2 Score : **-0.11**, which indicates poor model performance and suggests that the model needs tuning.

Model Performance for 2000 Epochs with Learning Rate = 0.5

```
In [16]: dg = GradientDescent(learning_rate = 0.5,epochs = 2000)
dg.fit(X_train,y_train)
y_pred = dg.predict(X_test)
print(f"RMSE --> {np.sqrt(mean_squared_error(y_test,y_pred))}")
print(f"R2 Score --> {r2_score(y_test,y_pred)}")
```

RMSE --> 55.06815880518703

R2 Score --> 0.4973745478804331

After increasing the epochs to 2000 and setting the learning rate to 0.5, the evaluation metrics are:

Root Mean Squared Error (RMSE): **55.068**

R2 Score: **0.4973**

The R2 score has improved significantly from **-0.11** to **0.4973**, indicating that the model is now fitting the data much better. This demonstrates that increasing the number of epochs and adjusting the learning rate can have a substantial impact on performance.

Scikit-Learn's Linear Regression Model

```
In [17]: lr = LinearRegression()
lr.fit(X_train,y_train)
y_pred = lr.predict(X_test)
print(f"RMSE --> {np.sqrt(mean_squared_error(y_test,y_pred))}")
print(f"R2 Score --> {r2_score(y_test,y_pred)}")
```

RMSE --> 54.91056597685105

R2 Score --> 0.5002472361041006

To compare the performance of our custom Gradient Descent-based Linear Regression, I used Scikit-Learn's LinearRegression model. The evaluation results are:

Root Mean Squared Error (RMSE): **54.910**

R2 Score: **0.5002**

These results are almost identical to our custom implementation, validating that our gradient descent approach is working correctly.

The above code demonstrates batch gradient descent

While Batch Gradient Descent is a fundamental optimization algorithm, it has several drawbacks:

- **low for Large Datasets** : since Batch Gradient Descent uses the entire dataset to compute the gradient in each iteration, it can be computationally expensive and slow for large datasets.
- **High Memory Usage** : Storing and processing the entire dataset in memory at once can be a problem, especially when working with big data.

And many more...

To overcome these disadvantages, two popular alternatives are:

Stochastic Gradient Descent (SGD)

Updates parameters using only one random sample per iteration. Faster but more noisy, leading to fluctuations in cost function reduction.

Mini-Batch Gradient Descent

Updates parameters using a small batch of data (e.g., 32 or 64 samples). Balances efficiency and accuracy.

Stochastic Gradient Descent (SGD) using Sklearn

```
In [18]: from sklearn.linear_model import SGDRegressor

sgd = SGDRegressor(max_iter=500, eta0=0.1, learning_rate='constant')

sgd.fit(X_train, y_train)
y_pred = sgd.predict(X_test)
print(f"RMSE --> {np.sqrt(mean_squared_error(y_test, y_pred))}")
print(f"R2 Score --> {r2_score(y_test, y_pred)}")
```

RMSE --> 56.26140744297447

R2 Score --> 0.47535619495029935

- Manually implemented Batch Gradient Descent took 2000 epochs to reach a valid solution.
- SGDRegressor from Scikit-learn achieved a similar result in just 500 epochs

While Batch Gradient Descent is a stable and straightforward optimization algorithm, its disadvantages, such as high computational expense and inefficiency with large datasets, make it less practical for many modern machine learning applications. Alternatives like Stochastic Gradient Descent and Mini-Batch Gradient Descent offer more efficient and flexible solutions for optimizing machine learning models

Conclusion

Implementing linear regression using gradient descent from scratch provides valuable insights into the inner workings of machine learning models. By coding the algorithm manually, we gain a deeper understanding of concepts like cost function optimization, learning rate selection, convergence criteria, and the impact of gradient updates on model performance.

Comparing the results with Scikit-Learn's built-in `LinearRegression` confirms the correctness of our implementation, as the RMSE and R2 scores are similar. While using libraries like Scikit-Learn is efficient and time-saving, coding from scratch enhances problem-solving skills, mathematical intuition, and debugging abilities—crucial for optimizing machine learning algorithms and understanding their limitations.

This hands-on approach strengthens foundational knowledge, making it easier to modify models for advanced applications, such as implementing different optimization techniques or extending to other regression problems.

Made With ❤️ by Syed Afnan